

Gestión de Contenedores en Entornos de Nube Híbrida

Sesión 1 · Fundamentos de contenedores y arquitectura de nube híbrida

Curso 101-EAN · 5 sesiones · Metodología teórico-práctica

Docente: Jawy Andrés Romero Pinto

MSc · Especialista en ciberseguridad y nube híbrida · LFCE

Objetivos de la sesión

Al final de estas 2 horas podrás...

Explicar

qué es un contenedor, en qué se diferencia de una máquina virtual y qué problema resuelve.

Reconocer

la arquitectura de nube híbrida y por qué los contenedores son su moneda de portabilidad.

Ejecutar

tus primeros contenedores con Docker: correr, listar, inspeccionar y eliminar.

Aplicar

el primer hábito de seguridad: usar imágenes oficiales verificadas.

«En mi máquina sí funciona»

El problema que llevó al mundo a los contenedores

El despliegue tradicional

- La app se desarrolla en un equipo con SUS versiones de todo.
- Al pasarla al servidor: otro sistema operativo, otras librerías, otra configuración.
- Resultado: errores que “solo pasan en producción”, despliegues de fin de semana, rollbacks dolorosos.
- Cada ambiente (desarrollo, pruebas, producción, nube) es un mundo distinto que hay que domar a mano.

Lo que necesitamos

- Que la aplicación viaje CON todo lo que necesita para correr.
- Que corra idéntica en el portátil, el servidor local y la nube.
- Que levante en segundos y se pueda replicar N veces.
- **Eso, exactamente, es un contenedor.**

De los servidores físicos a los contenedores

Tres generaciones de aprovechar el mismo hardware

Servidor físico

- 1 app por servidor
- Semanas para aprovisionar
- Capacidad desperdiciada
- Escalar = comprar hierro

Máquinas virtuales

- Varias “máquinas” por servidor
- Cada VM: SO completo (GBs)
- Minutos para arrancar
- Mejor uso, aún pesado

Contenedores

- Decenas por servidor
- Comparten el kernel del host (MBs)
- Arrancan en segundos
- Portables entre local y nube

La flecha de la evolución: más densidad, más velocidad, más portabilidad.

¿Qué es un contenedor?

Un proceso (o grupo de procesos) aislado que empaca la aplicación junto con TODO lo que necesita: librerías, dependencias, configuración y sistema de archivos.

Qué SÍ lleva adentro

- El código de la aplicación
- Sus librerías y dependencias exactas
- Configuración y variables
- Un mini sistema de archivos

Qué NO lleva adentro

- Un sistema operativo completo (usa el kernel del host)
- Datos permanentes (por defecto es efímero — S4)
- La configuración del servidor donde corre

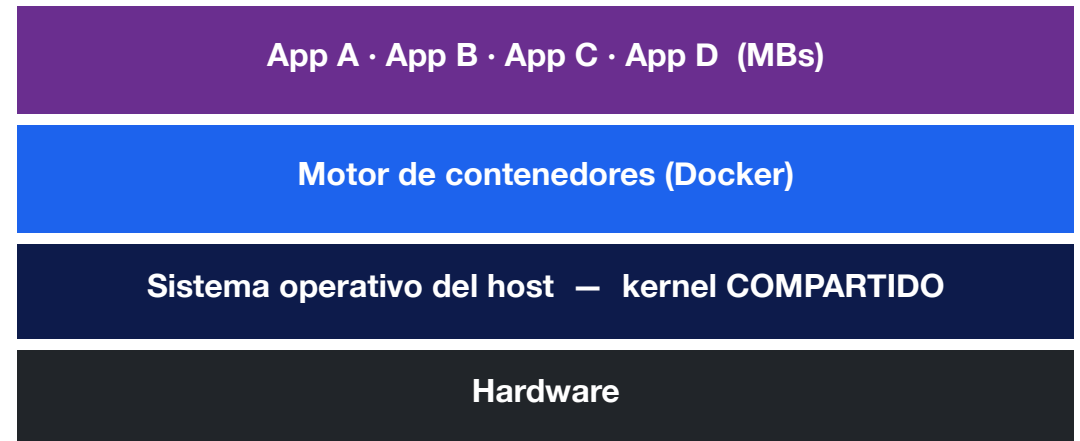
Contenedor vs. máquina virtual

La diferencia está en qué se comparte

Máquinas virtuales



Contenedores



La VM virtualiza el hardware (cada una trae su SO). El contenedor virtualiza el SO (todos comparten el kernel). Por eso pesa MBs y arranca en segundos.

Imagen vs. contenedor

El concepto más importante de hoy

IMAGEN — la receta

- Plantilla inmutable: app + dependencias + configuración.
- Se construye una vez, se versiona (etiquetas o tags).
- Se guarda y comparte en un registro (Docker Hub).
- Ej.: `nginx:1.27`, `postgres:16`

CONTENEDOR — el plato servido

- Instancia VIVA de una imagen: un proceso corriendo.
- De una misma imagen puedes servir N contenedores.
- Se crea y se destruye en segundos, sin afectar la imagen.
- `docker run nginx:1.27` → **nace un contenedor**

Analogía: la imagen es la receta congelada; el contenedor es el plato servido. Muchos platos de la misma receta — todos idénticos.

Cómo funciona Docker

Tres piezas que hoy vas a tocar con tus manos



El flujo de hoy en el lab:

1) escribes `docker run nginx` → 2) el motor busca la imagen local; si no está, la baja del registro (pull) → 3) crea el contenedor y lo arranca. Todo en segundos.

Docker Hub: la biblioteca de imágenes

Y el primer hábito de seguridad del curso

Qué encuentras en el Hub

- Millones de imágenes listas: nginx, postgres, python, redis...
- Imágenes oficiales: mantenidas y verificadas (sello «Docker Official Image»).
- Verified publishers: del fabricante (Microsoft, AWS...).
- Comunidad: cualquiera puede publicar — y ahí está el riesgo.



Eje de seguridad · S1

Regla de hoy: en este curso SOLO usamos imágenes oficiales o de publicadores verificados.

- Una imagen de origen desconocido puede traer malware o mineros de criptomonedas.
- Verifica el sello en el Hub antes del pull — lo harás en el lab.
- *Para una entidad financiera, esta regla no es opcional: es política.*

¿Qué es la nube híbrida?

El contexto real de Colpensiones



Infraestructura local (on-premise)

- Servidores propios, en el datacenter de la entidad.
- Control total, datos sensibles en casa.
- Capacidad fija: crecer toma meses.



Nube pública (AWS, Azure...)

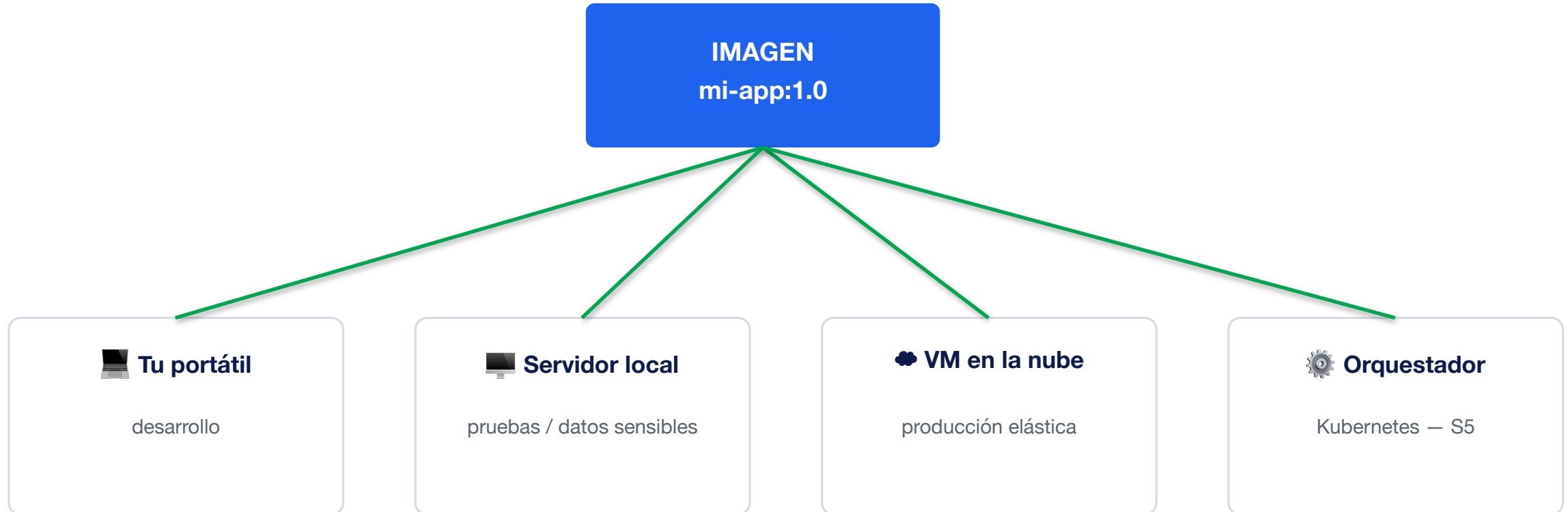
- Capacidad elástica: crece y decrece por demanda.
- Pagas por uso, sin comprar hardware.
- Servicios gestionados listos para usar.

Nube híbrida = usar ambos, con criterio.

Lo sensible y estable en casa; lo elástico y expuesto en la nube. El reto: que las aplicaciones puedan MOVERSE entre los dos mundos sin reescribirse. Ese movimiento es exactamente lo que el contenedor hace posible.

El contenedor: la moneda de la nube híbrida

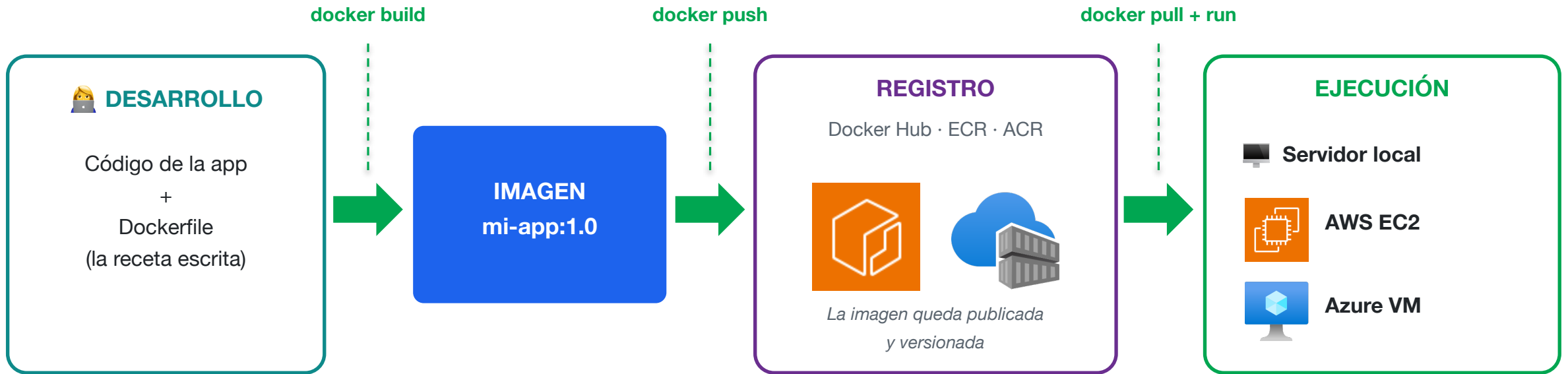
Misma imagen, cualquier destino



docker run se comporta IGUAL en los cuatro destinos. La imagen no se toca: se construye una vez y corre donde convenga. Eso es portabilidad local ↔ nube.

Flujo de infraestructura: del código al contenedor corriendo

El camino que recorre toda aplicación contenerizada



Cuatro etapas, tres comandos. La imagen se construye UNA vez y el registro la reparte a cualquier destino — local o nube. Hoy harás pull+run; en la S2 harás build+push de tu propia imagen.

Arquitectura de referencia: contenedores en nube híbrida

Así se ve el destino al que apunta este curso



Cada carga vive donde más conviene — y como todas están contenerizadas, moverlas de un lado al otro es un docker pull, no un proyecto de migración de seis meses.

Dónde corren los contenedores en Azure

☁ Nuestro entorno activo

La nube de ESTE grupo — sus cuentas de Azure ya están activas



Azure Container Registry

El registro privado de imágenes en Azure — equivalente a ECR.

→ Guardar y versionar TUS imágenes



Azure Virtual Machine + Docker

Una VM donde instalas Docker — idéntico a EC2 + Docker.

→ Lo que haremos en este curso (S5)



Azure Container Instances

Contenedores sueltos por demanda, sin gestionar servidores — el "Fargate simple" de Azure.

→ Tareas puntuales y pruebas rápidas



Azure Container Apps

Apps contenerizadas gestionadas con escalado automático (Kubernetes por debajo, sin verlo).

→ Producción sin administrar clúster



Azure Kubernetes Service

Kubernetes gestionado por Azure — equivalente a EKS.

→ El estándar de orquestación (S5)

Ojo: Docker Hub NO es de Azure — es el registro público y neutral de Docker. ACR es el registro privado DE Azure. En el lab usamos Docker Hub; la nube del curso es Azure.

Dónde corren los contenedores en AWS

El mapa equivalente en AWS — mismo menú, otros nombres



Amazon ECR

Elastic Container Registry — el registro privado de imágenes de tu organización.

→ Guardar y versionar TUS imágenes



Amazon EC2 + Docker

Una máquina virtual donde instalas Docker y corres contenedores a mano.

→ Lo que haremos en este curso (S5)



Amazon ECS

Elastic Container Service — el orquestador nativo de AWS: tú defines el servicio, él mantiene los contenedores vivos.

→ Producción sin administrar Kubernetes



AWS Fargate

Contenedores serverless: ni siquiera ves los servidores — pagas por contenedor corriendo.

→ Cargas variables sin gestionar VMs



Amazon EKS

Elastic Kubernetes Service — Kubernetes gestionado por AWS.

→ El estándar de orquestación (S5)

Todos consumen las MISMAS imágenes Docker que aprenderás a construir — cambia quién administra, no el contenedor.

Casos de uso reales

Quién usa contenedores y para qué



Microservicios a escala global

Netflix y Spotify parten su plataforma en cientos de servicios pequeños, cada uno en contenedores, desplegados miles de veces por semana sin tumbar el servicio.



Modernización en banca

Bancos como Capital One e ING contenerizaron aplicaciones críticas: despliegues que tomaban semanas pasaron a horas, con ambientes idénticos y auditables.



Picos de demanda estacionales

Comercios en temporada alta replican contenedores en la nube y luego los apagan. Piensa en el fin de mes de pagos de una entidad de pensiones: misma lógica.



Ambientes efímeros

Equipos de desarrollo y capacitación levantan ambientes completos en minutos y los destruyen al terminar — exactamente lo que tú harás hoy en el lab.

El patrón común: empacar una vez, desplegar donde convenga, escalar y destruir sin miedo.

Nuestra plataforma de práctica: Play with Docker

Docker real, en el navegador, gratis

¿Qué es?

- labs.play-with-docker.com — terminales Linux con Docker preinstalado, en tu navegador.
- Solo necesitas una cuenta gratuita de Docker Hub.
- Cero instalación: todos trabajamos en el mismo ambiente.
- Cada sesión dura máximo 4 horas y se borra al cerrar.

Reglas de juego

- Guarda tu bitácora FUERA de la plataforma (se borra todo al expirar).
- Si la sesión muere: crea otra instancia y repite — en contenedores eso toma segundos.
- Es un ambiente compartido de práctica: nada de datos reales.
- Plan B del curso: Docker Desktop en tu equipo (opcional).

Los 7 comandos de hoy

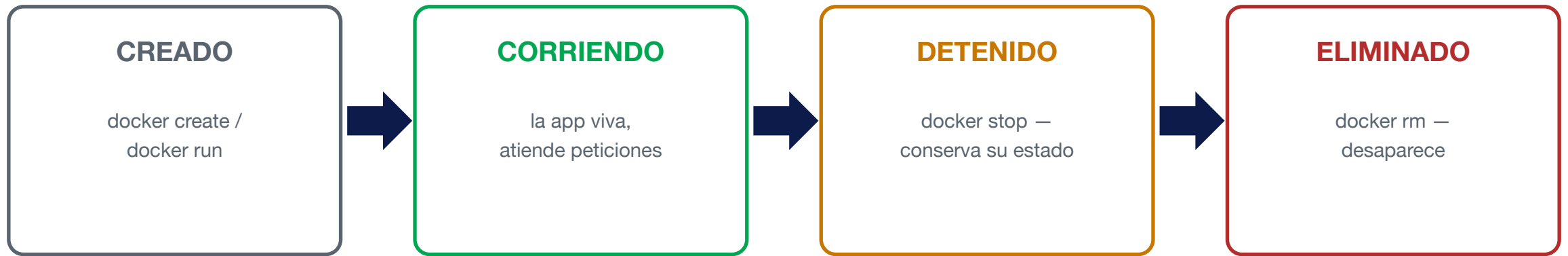
Tu vocabulario mínimo viable de Docker

<code>docker run <imagen></code>	Crea y arranca un contenedor desde una imagen (baja la imagen si no está).
<code>docker ps</code> / <code>docker ps -a</code>	Lista contenedores corriendo / todos (incluye detenidos).
<code>docker images</code>	Lista las imágenes descargadas en la máquina.
<code>docker pull <imagen></code>	Descarga una imagen del registro sin ejecutarla.
<code>docker logs <contenedor></code>	Muestra la salida (logs) de un contenedor.
<code>docker stop <contenedor></code>	Detiene un contenedor en ejecución.
<code>docker rm <contenedor></code> / <code>docker rmi <imagen></code>	Elimina un contenedor / una imagen.

Truco: `docker run -d` = en segundo plano · `docker run -p 8080:80` = publicar puerto (lo usamos hoy).

Ciclo de vida de un contenedor

Nace, corre, muere — y no pasa nada



⚠ El contenedor es EFÍMERO por diseño.

Lo que escribas dentro se pierde al eliminarlo — hoy lo vas a comprobar en el lab. ¿Y los datos que deben sobrevivir? Ese problema lo resolvemos en la Sesión 4 con volúmenes y respaldos.

El mapa del curso

Hoy pones el cimiento de todo lo demás

S1

HOY · Fundamentos y arquitectura híbrida

Primer contenedor + bitácora

S2

Imágenes propias: construir y publicar

Tu imagen en Docker Hub

S3

Conectividad y redes

App multi-contenedor en red aislada

S4

Persistencia y respaldo

Volumen + respaldo restaurado · inicia mini-proyecto

S5

Portabilidad y orquestación

Mini-proyecto: local → nube + demo Kubernetes



Eje transversal en las 5 sesiones: buenas prácticas de seguridad de contenedores.

El laboratorio de hoy

60 minutos · Play with Docker · en parejas o individual

Parte 1 · Tu primer contenedor (20 min)

Entra a Play with Docker, corre hello-world y luego un servidor web nginx real con puerto publicado. Míralo responder en el navegador.

Parte 2 · Explora y comprueba lo efímero (25 min)

Entra a un contenedor de Ubuntu, crea un archivo, destrúyelo y comprueba que el archivo se fue. Verifica el sello de imagen oficial en Docker Hub antes de cada pull.

Parte 3 · Bitácora — tu entregable (15 min)

Documenta comandos, capturas y 3 reflexiones en la plantilla de bitácora. Es la evidencia práctica de la sesión y se entrega al cierre.

Entregable de la S1: primer contenedor corriendo (captura) + bitácora diligenciada.

Las 3 ideas que te llevas hoy

1 · El contenedor empaca la aplicación con todo lo suyo: corre idéntica en local y en la nube.

2 · La imagen es la receta (inmutable); el contenedor es la instancia viva (efímera).

3 · En nube híbrida, el contenedor es la moneda de portabilidad — y siempre de imagen oficial.

Próxima sesión: construyes TU propia imagen y la publicas en Docker Hub.